

Gasverbrauch-Statistik in ioBroker

Verbrauch, Durchfluss, abrechnungsfähiger Zählerstand und automatische Abrechnung per Telegram, E-Mail und PDF - aus dem HmIP-ESI-GAS

Datei	gasverbrauch_statistik_v3.0.0.js
Version	3.0.0
Stand	23.09.2026
Adapter	javascript (Typ Javascript/ECMAScript)
Erstellt für	ssbingo
Benötigt für	ioBroker.vis-2-widgets-wolf (Gaszähler-Widget)

1. Ausgangslage

Der **HmIP-ESI-GAS** zeigt in der CCU-WebUI vier Statistikwerte an – *Heute, Gestern, Vergangene 7 Tage, Vergangene 30 Tage*. Diese Werte sind **keine Gerätedatenpunkte**: Sie werden CCU-intern aus den gespeicherten Zählerständen berechnet und liegen als Metadaten vor. Über `hm-rpc` kommen sie deshalb grundsätzlich nicht in ioBroker an.

In ioBroker verfügbar sind:

Quelle	Datenpunkt	Inhalt
hm-rpc	GAS_FLOW , GAS_FLOW_STATUS	aktueller Gasfluss in m³/h
hm-rpc	GAS_VOLUME , GAS_VOLUME_STATUS	Volumen in m³ – kein Gesamtzählerstand
hm-rega	svEnergyCounter...<Seriennummer>	Zählerstand – versteckte Systemvariable

Der Zählerstand ist monoton steigend und damit die einzige Größe, die man wirklich braucht: Alles andere lässt sich daraus ableiten. Genau das macht dieses Skript – unabhängig von CCU-Firmware-Internia, die HomeMatic jederzeit ändern kann.

Hinweis zur versteckten Systemvariablen: Damit `svEnergyCounter...` in ioBroker erscheint, muss im Adapter **hm-rega** unter *Synchronisieren* die Option für **nicht sichtbare Variablen** aktiviert werden. Danach die Instanz neu starten und im Objektbaum nach der Seriennummer des Geräts filtern.

2. Installation

1. Im Adapter **javascript** ein neues Skript vom Typ *Javascript/ECMAScript* anlegen.
2. Inhalt von `gasverbrauch_statistik.js` einfügen.

3. Im Konfigurationsblock oben mindestens `SRC` anpassen (siehe Abschnitt 3).
4. Skript starten und das Log prüfen – die States werden automatisch angelegt.

Voraussetzungen:

- Adapter `javascript` (JavaScript-Engine, Node ≥ 18)
- Objektbaum `0_userdata.0` (Standard; anderer Pfad über `PFAD` möglich)
- optional: ein History-Adapter für die einmalige Vorbefüllung (Abschnitt 5)
- für die Berichte: Adapter `telegram` und/oder `email` (Abschnitt 11)
- für das PDF: npm-Modul **pdfkit**, einzutragen in den Einstellungen der javascript-Instanz unter *Zusätzliche NPM-Module*. Fehlt es, laufen Telegram und E-Mail weiter – nur ohne Anhang.

3. Konfiguration

Alle Einstellungen stehen im Kopf des Skripts.

Konstante	Default	Bedeutung
<code>SRC</code>	<code>'hm-rega.0.12345'</code>	Pflichtangabe. State mit dem Zählerstand in m ³ – die Rega-Systemvariable <code>svEnergyCounter...</code> (hm-rpc liefert für den ESI-GAS keinen Gesamtzählerstand)
<code>PFAD</code>	<code>'0_userdata.0.Gas'</code>	Zielordner für die erzeugten States
<code>INKL_HEUTE</code>	<code>false</code>	<code>false</code> = 7-/30-Tage-Summe zählt nur abgeschlossene Tage (Verhalten der CCU). <code>true</code> = laufender Tag wird mitgezählt
<code>TAGE_HISTORIE</code>	<code>70</code>	Anzahl vorgehaltener Tageswerte. Für <i>Letzter Monat</i> sind mindestens 62 nötig
<code>NK</code>	<code>3</code>	Nachkommastellen der Verbrauchswerte
<code>HISTORY_INSTANCE</code>	<code>'influxdb.0'</code>	Instanz für die Vorbefüllung. <code>''</code> schaltet die Funktion ab. Auch <code>'history.0'</code> oder <code>'sql.0'</code>
<code>BACKFILL_BEIM_START</code>	<code>true</code>	Beim allerersten Start automatisch aus der History vorbe füllen
<code>KORREKTUR_AKTIV</code>	<code>true</code>	Abgleich auf abgelesene Zählerstände (Abschnitt 7)
<code>FAKTOR_AUTO</code>	<code>true</code>	Korrekturfaktor automatisch aus je zwei Ablesungen bilden
<code>FAKTOR_MANUELL</code>	<code>1</code>	fester Faktor, wenn <code>FAKTOR_AUTO = false</code>
<code>FAKTOR_MIN</code> / <code>FAKTOR_MAX</code>	<code>0.8 / 1.25</code>	Plausibilitätsgrenzen für den Faktor
<code>FAKTOR_MIN_VERBRAUCH</code>	<code>2</code>	Mindestverbrauch in m ³ zwischen zwei Ablesungen für einen neuen Faktor

Konstante	Default	Bedeutung
ABLESUNG_MAX_SPRUNG	5	Ablesungen, die weiter vom erwarteten Stand abweichen, werden abgelehnt
DURCHFLUSS_AKTIV	true	Durchfluss aus der Zähleränderung berechnen (Abschnitt 8)
FENSTER_MIN	10	Glättungsfenster in Minuten. Größer = ruhiger, aber träger
NULL_NACH_MIN	10	Ohne Zähleränderung für so viele Minuten wird der Durchfluss 0
LUECKE_MIN	7	Abstand zweier Zähleränderungen, ab dem eine Pause angenommen wird ($\leq$ NULL_NACH_MIN)
ANLAUF_MIN	3	Angenommene Dauer der ersten Änderung nach einer Pause ($\approx$ Sendeintervall des ESI)
MAX_DURCHFLUSS	10	Plausibilitätsgrenze in m³/h; höhere Werte werden begrenzt und protokolliert
BRENNWERT	11.2	kWh je m³ laut Gasrechnung
ZUSTANDSZAHL	0.95	Z-Zahl laut Gasrechnung
ARBEITSPREIS_CT_KWH	8.90	Arbeitspreis in ct/kWh, netto
GRUNDPREIS_EUR_MONAT	12.50	Grundpreis in €/Monat, netto
MWST_PROZENT	19	Mehrwertsteuersatz
ABSCHLAG_EUR	120.00	monatlicher Abschlag brutto; 0 = keine Abschlagsverrechnung
KUNDE	Objekt	Name, Anschrift, Lieferant, Tarif, Vertragskonto, Zählernummer für Kopfzeile und PDF
TELEGRAM_INSTANZ	'telegram.0'	' ' schaltet Telegram ab
TELEGRAM_USER	' '	Empfänger; leer = alle registrierten Nutzer
TAGESBERICHT_AKTIV	true	Tagesübersicht per Telegram
TAGESBERICHT_CRON	'1 0 * * *'	00:01 Uhr – Bilanz des abgeschlossenen Vortags
MONATSBERICHT_AKTIV	true	Monatsabrechnung per Telegram und E-Mail
MONATSBERICHT_CRON	'10 0 1 * *'	am 1. um 00:10 Uhr für den abgeschlossenen Vormonat
EMAIL_INSTANZ	'email.0'	' ' schaltet den E-Mail-Versand ab
EMAIL_VON / EMAIL_AN	' ' / Adresse	Absender (leer = Instanz-Standard) und Empfänger
PDF_AKTIV	true	PDF der Monatsabrechnung erzeugen
PDF_PFAD	/opt/iobroker/ iobroker-data/ gasabrechnung	Ablageort; muss für ioBroker beschreibbar sein

Konstante	Default	Bedeutung
PDF_SCHRIFT / PDF_SCHRIFT_FETT	DejaVu-Pfade	Schriftdateien für das PDF; fehlen sie, wird Helvetica genutzt
PDF_PER_TELEGRAM	true	PDF zusätzlich als Telegram-Dokument
SYMBOLE	true	dezente Symbole in den Nachrichten
DEBUG	false	Zusätzliche Log-Ausgaben bei jeder Berechnung

4. Erzeugte States

Alle unterhalb von `PFAD` (Default `0_userdata.0.Gas`):

State	Typ	Rolle	Inhalt
Zaehlerstand	number	<code>value.volume</code>	gespiegelter Zählerstand in m ³
Heute	number	<code>value</code>	Verbrauch seit Mitternacht
Gestern	number	<code>value</code>	Verbrauch des Vortags
Letzte7Tage	number	<code>value</code>	Summe der letzten 7 Tage
Letzte30Tage	number	<code>value</code>	Summe der letzten 30 Tage
DieserMonat	number	<code>value</code>	Summe seit Monatsbeginn inkl. heute
LetzterMonat	number	<code>value</code>	Summe des Vormonats
Basis	number	<code>value.volume</code>	Zählerstand um Mitternacht (intern)
BasisDatum	string	<code>date</code>	Datum dieser Basis, <code>YYYY-MM-DD</code> (intern)
Historie	string	<code>json</code>	Tageswerte als JSON-Ringpuffer (intern)
Backfill	boolean	<code>button</code>	löst die Vorbefüllung aus (Abschnitt 5)
Ablesung	number	<code>value</code>	Eingabe: hier den am Zähler abgelesenen Stand eintragen
ZaehlerstandRoh	number	<code>value.volume</code>	Wert der CCU, unkorrigiert
LetzteAblesung	number	<code>value.volume</code>	zuletzt übernommene Ablesung
LetzteAblesungsZeit	number	<code>value.time</code>	Zeitpunkt dieser Ablesung

State	Typ	Rolle	Inhalt
Korrekturfaktor	number	value	aktiver Faktor (1 = keine Korrektur)
AbweichungLetzteAblesung	number	value	Korrektur in m³ beim letzten Abgleich
AbweichungProzent	number	value	Impulsverlust in %
Anker	string	json	Ankerpunkt (intern)
Durchfluss	number	value	berechneter Durchfluss in m³/h (Abschnitt 8)
VerbrauchAktiv	boolean	indicator	true, solange Gas fließt
ZaehlerLetzteAenderung	number	value.time	Zeitstempel der letzten Zähleränderung (ms)
Kosten.EnergieHeute	number	value.power.consumption	kWh heute
Kosten.EnergieMonat	number	value.power.consumption	kWh im laufenden Monat
Kosten.KostenHeute	number	value	Kosten heute, brutto
Kosten.KostenMonat	number	value	Kosten im laufenden Monat, brutto
Kosten.PrognoseMonat	number	value	hochgerechnete Monatskosten
Kosten.SaldoJahr	number	value	Abschläge minus Kosten im laufenden Jahr
Bericht.Monatsstaende	string	json	Zählerstände zum Monatsanfang (intern)
Bericht.Monatsarchiv	string	json	abgerechnete Monate (intern)
Bericht.TagesberichtSenden	boolean	button	Tagesbericht von Hand senden
Bericht.MonatsberichtSenden	boolean	button	Monatsbericht des Vormonats von Hand senden
Bericht.LetzterMonatsbericht	string	text	zuletzt abgerechneter Monat
Bericht.LetztePdfDatei	string	text	Pfad der zuletzt erzeugten PDF-Datei
Bericht.MonatFuerBericht	string	text	Monat YYYY-MM für den manuellen Bericht; leer = Vormonat

Die internen States (`Basis` , `BasisDatum` , `Historie` , `Anker`) nicht von Hand ändern – sie bilden den Zustand ab, aus dem alle anderen Werte berechnet werden.

Format der Historie:

```
[{"d": "2026-09-18", "v": 1.3}, {"d": "2026-09-19", "v": 1.6}]
```

Ein Eintrag je **abgeschlossenem** Tag. Der laufende Tag steht nicht darin, sondern ergibt sich aus `Zaehlerstand - Basis` .

5. Vorbefüllung aus dem History-Adapter

Ohne Vorbefüllung sind die 7-/30-Tage-Werte erst nach 7 bzw. 30 Tagen Laufzeit vollständig. Wird der Zählerstand bereits in `influxdb` , `history` oder `sql` aufgezeichnet, baut das Skript die Historie daraus in einem Rutsch auf.

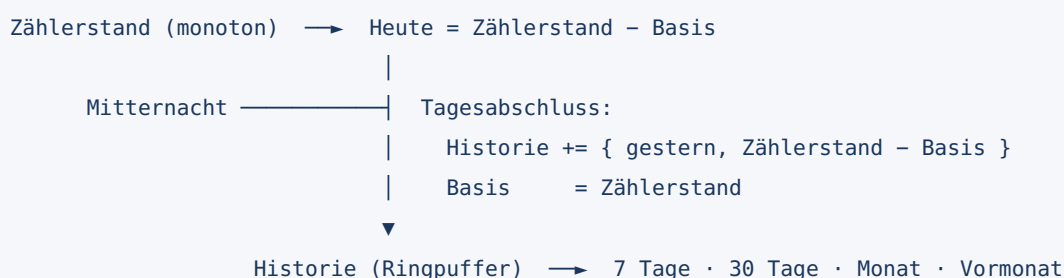
Automatisch: beim allerersten Start, sofern `HISTORY_INSTANCE` gesetzt und `BACKFILL_BEIM_START = true` ist.

Manuell: den State `<PFAD>.Backfill` auf `true` setzen (Objektbaum oder VIS-Button). Die Historie wird dabei komplett neu aufgebaut.

Verfahren: Für jede Tagesgrenze (00:00 Uhr) wird der Zählerstand per `getHistory` mit `aggregate: 'max'` über ein Fenster davor abgefragt – beim monoton steigenden Zähler ist das Maximum des Fensters genau der Stand zur Tagesgrenze. Der Tagesverbrauch ist die Differenz zweier aufeinanderfolgender Tagesgrenzen. Pro Tagesgrenze fällt nur eine Abfrage mit einem Rückgabewert an; auch bei minütlichem Logging bleibt die Last gering. Liefert das erste Fenster (2 Tage) nichts, wird auf 7 und dann 30 Tage erweitert.

Tage, für die keine History-Daten vorliegen, werden ausgelassen statt mit falschen Werten gefüllt; das Log nennt die Anzahl. Liefert die History gar nichts, bricht die Vorbefüllung mit einer Warnung ab und der laufende Betrieb beginnt bei null – ein typischer Grund ist, dass der Quell-State im History-Adapter nicht zur Aufzeichnung aktiviert ist.

6. Funktionsweise im laufenden Betrieb



Auslöser der Berechnung

Auslöser	Zweck
Änderung von SRC	laufende Aktualisierung von Heute
<code>schedule('1 0 * * *')</code>	Tagesabschluss um 00:01 Uhr
<code>schedule('5 * * * *')</code>	stündliches Sicherheitsnetz
<code>schedule('* * * * *')</code>	minütliche Prüfung: Durchfluss auf 0, wenn der Zähler steht
Ablesung beschrieben	Abgleich auf den abgelesenen Zählerstand
<code>Backfill = true</code>	Vorbefüllung aus der History
TAGESBERICHT_CRON	Tagesübersicht per Telegram
MONATSBERICHT_CRON	Monatsabrechnung per Telegram, E-Mail und PDF

Überlappende Läufe sind durch eine Sperre ausgeschlossen – wichtig, weil die Vorbefüllung länger dauern kann als das Intervall des Zählers. Der Durchfluss wird außerhalb dieser Sperre sofort bei jeder Zähleränderung aktualisiert.

Robustheit

- **Verpasster Mitternachtslauf:** Der Tagesabschluss hängt nicht allein am Cron. Bei jeder Zähleränderung wird geprüft, ob sich das Datum gegenüber `BasisDatum` geändert hat, und der Abschluss gegebenenfalls nachgeholt.
- **Ausfalltage:** War ioBroker mehrere Tage aus, werden die fehlenden Tage mit `0` aufgefüllt, damit die Reihe nicht verrutscht. Der in dieser Zeit angefallene Verbrauch wird dem ersten Tag der Lücke zugeschlagen – die Summen über 7 und 30 Tage bleiben dadurch korrekt, die einzelnen Tageswerte innerhalb der Lücke nicht.
- **Zählerreset / Zählerwechsel:** Sinkt der Zählerstand, wird die Basis zurückgesetzt und der betroffene Tag mit `0` gewertet, statt negative Werte zu erzeugen. Eine Warnung landet im Log.
- **Ungültige Werte:** `null`, `undefined` und nicht-numerische Werte werden ignoriert, nicht als 0 verarbeitet.

7. Abgleich auf den echten Zähler

Der ESI zählt Impulse. Geht ein Impuls verloren, fehlt er dauerhaft – der Stand in der CCU wächst langsamer als der echte Zähler, und der Fehler summiert sich. Im vorliegenden Fall fehlten über fünf Tage 0,321 m³ bei 5,391 m³ echtem Verbrauch, also rund 6 %.

Das Skript gleicht das über **Ankerpunkte** aus: Du trägst einen abgelesenen Zählerstand ein, und ab da gilt

```
korrigiert = Ablesung(Anker) + Faktor * (Rohwert - Rohwert(Anker))
```

Aus je zwei aufeinanderfolgenden Ablesungen ergibt sich der Faktor:

Faktor = echter Verbrauch / von der CCU gezählter Verbrauch

Im Beispiel: 3,190 m³ echt zu 3,000 m³ gezählt ergibt 1,0633, also 6,3 % Verlust.

Ablauf

1. Am Zähler ablesen und den Wert **sofort** in `0_userdata.0.Gas.Ablesung` eintragen (Objektbaum oder VIS-Eingabefeld).
2. Der korrigierte Zählerstand springt exakt auf diesen Wert. Die Höhe der Korrektur steht in `AbweichungLetzteAblesung`.
3. Ab der **zweiten** Ablesung wird zusätzlich der Faktor gebildet und ab dann laufend angewendet. `AbweichungProzent` zeigt den Impulsverlust.
4. Jede weitere Ablesung führt den Faktor nach. Je länger der Abstand, desto stabiler der Wert – zwischen zwei Ablesungen sollten mindestens `FAKTOR_MIN_VERBRAUCH` m³ liegen.

Was das Skript prüft

Prüfung	Verhalten
Ablesung kleiner als die vorige	abgelehnt, Warnung im Log
Abweichung größer als <code>ABLESUNG_MAX_SPRUNG</code> (5 m ³)	abgelehnt – schützt vor Tippfehlern
Faktor außerhalb 0,8–1,25	alter Faktor bleibt, Warnung im Log
Rohwert der CCU fällt (Offset geändert, Gerätetausch)	Anker wird nachgezogen, der korrigierte Stand bleibt stetig

Auswirkung auf die Tageswerte: Beim Abgleich wird die Tagesbasis um denselben Betrag mitgezogen. *Heute* bleibt dadurch unverändert – die nachträgliche Korrektur wird also nicht rückwirkend auf einzelne Tage verteilt. Das ist bewusst so: Die Tageswerte bleiben nachvollziehbar, die Summe stimmt ab dem Abgleich wieder.

Für die Abrechnung

- Maßgeblich für die Abrechnung ist `Zaehlerstand` (korrigiert), nicht `ZaehlerstandRoh`.
- Zu jedem Abrechnungsstichtag eine Ablesung eintragen. Dann steht der Wert zum Stichtag exakt auf dem echten Zähler, und die Korrektur der Zwischenzeit ist abgeschlossen.
- Zwischen zwei Ablesungen ist der Wert eine **Hochrechnung**. Der Faktor gleicht den durchschnittlichen Impulsverlust aus, nicht den einzelnen verlorenen Impuls. Kurzfristig bleibt eine Restabweichung von einigen Zehntel Prozent.
- Der HmIP-ESI ist kein eichfähiges Messmittel. Rechtlich verbindlich ist immer der Zählerstand am Zähler selbst. Das Skript liefert die Werte dazwischen und macht die Abweichung sichtbar – es ersetzt die Ablesung zum Stichtag nicht.

Offset in der CCU: Nach dem ersten Abgleich den Startwert in den Geräteeinstellungen der CCU **nicht mehr ändern**. Die Korrektur passiert ausschließlich im Skript; eine zusätzliche Änderung in der CCU würde als Verbrauch gewertet.

8. Berechneter Durchfluss

Warum nicht einfach GAS_FLOW ? Der ESI zählt nur Impulse. Solange Impulse kommen, sendet er im Rhythmus $(A + 1) \times C$ (Abschnitt 9). Hört der Verbrauch auf, kommen keine Impulse mehr – das Gerät hält seinen Zustand für *unverändert* und meldet den Durchfluss 0 erst mit der nächsten unveränderten zyklischen Meldung, mit Standardwerten nach 1,4 bis 2,2 Stunden. Bis dahin steht in GAS_FLOW der letzte Wert, obwohl längst kein Gas mehr fließt.

Das Skript rechnet den Durchfluss deshalb selbst aus dem Zählerstand und setzt ihn nach NULL_NACH_MIN Minuten ohne Zähleränderung zuverlässig auf 0.

Verfahren

- Jede Zähleränderung ist eine Probe (*Zeit, Stand*).
- **Normalfall:** Durchfluss = $\Delta \text{Stand} / \Delta \text{Zeit}$ zwischen der neuesten Probe und einem Anker am Beginn des Glättungsfensters (FENSTER_MIN).
- **Anlauf nach Pause:** Liegt die vorige Änderung länger als LUECKE_MIN zurück, enthielte ΔZeit die ganze Pause und der Wert liefe langsam hoch. Stattdessen wird angenommen, dass der Verbrauch innerhalb von ANLAUF_MIN angefallen ist – das entspricht einem Sendeintervall des ESI.
- **Stillstand:** Eine minütliche Prüfung setzt 0, sobald NULL_NACH_MIN keine Änderung kam.
- **Zählerreset / Sprung:** Sinkt der Stand, beginnt die Berechnung neu (Wert 0). Werte über MAX_DURCHFLUSS werden begrenzt und im Log gemeldet.
- **Neustart:** Nach einem Skript-Neustart steht der Durchfluss auf 0, bis die nächste Änderung kommt; der letzte Stand wird aus dem Quell-State übernommen.

Testergebnis (Sandbox mit simulierter Uhr, ESI mit A = 0):

Szenario	Eingabe	Ergebnis
Brenner läuft	+0,06 m³ alle 3 min (1,2 m³/h)	1,2 m³/h ab der ersten Meldung
Brenner aus	keine Änderung	nach 9 min noch 1,2 – nach 11 min 0
Warmwasser nach 40 min Pause	+0,03 m³ alle 3 min (0,6 m³/h)	0,6 m³/h sofort
Kleinlast	+0,01 m³ alle 6 min (0,1 m³/h)	0,1 m³/h (Übergang über ein Fenster)
Unplausibler Sprung	+5 m³ in 1 min	auf 10 m³/h begrenzt, Warnung im Log
Zählerreset	Stand fällt auf 0,5	0 m³/h, Neubeginn

Abstimmung der Parameter

Wunsch	Einstellung
0 schneller anzeigen	NULL_NACH_MIN kleiner – aber größer als der Impulsabstand bei kleinster Last lassen, sonst springt die Anzeige bei Kleinlast zwischen 0 und Wert
ruhigere Anzeige	FENSTER_MIN größer
CCU mit Standardwerten (A = 1)	ANLAUF_MIN = 6 , LUECKE_MIN = 10 , NULL_NACH_MIN = 15

Die Genauigkeit ist durch die Impulswertigkeit begrenzt: Bei 0,01 m³/Impuls und 0,1 m³/h kommt nur alle 6 Minuten ein Impuls – feiner lässt sich kleine Last grundsätzlich nicht auflösen.

Aktualität der Quelle: Der Zählerstand kommt als Rega-Systemvariable über hm-rega. Änderungen erreichen ioBroker daher erst mit dem nächsten Abfragezyklus der hm-rega-Instanz (üblich 30 s). Gegenüber dem Sendeintervall des ESI von 2–3 Minuten ist das unkritisch; für sofortige Übernahme siehe *Polling-Trigger* in Abschnitt 9.

9. Empfohlene CCU-Einstellungen

Der Durchfluss im Skript kann nur so aktuell sein wie die Zählerstände, die der ESI liefert. Die Sendehäufigkeit von Homematic-IP-Geräten ergibt sich aus drei Werten:

	Parameter (WebUI)	Standard
A	Anzahl der auszulassenden Statusmeldungen	1
B	Anzahl der auszulassenden, unveränderten Statusmeldungen	20
C	Grundintervall (vom Hersteller fest)	ca. 120–184 s

- bei Verbrauch (Stand ändert sich): Meldung alle $(A + 1) \times C$
- ohne Verbrauch (Stand unverändert): Meldung alle $(A + 1) \times (B + 1) \times C$

Variante	A	B	Meldung bei Verbrauch	ohne Verbrauch
Empfehlung	0	5	alle 2–3 min	ca. 12–18 min
maximal aktuell	0	0	alle 2–3 min	alle 2–3 min
Standard	1	20	alle 4–6 min	ca. 1,4–2,2 h

Einstellung in der CCU: *Einstellungen* → *Geräte* → *HmIP-ESI* → *Einstellen*, bei allen Kanälen, die die Parameter anbieten. **„Zyklische Statusmeldung“ muss aktiviert sein** – A und B regeln nur deren Häufigkeit; ohne sie meldet das Gerät nur bei Änderung, und die Erreichbarkeitsüberwachung der CCU verliert ihre Grundlage.

Nach dem Speichern steht das Gerät auf *Konfiguration ausstehend*. Ein kurzer Druck auf die Systemtaste des ESI überträgt die Konfiguration sofort.

hm-rega: Abfrageintervall und Polling-Trigger

hm-rpc liefert für den ESI-GAS keinen Gesamtzählerstand; die Quelle ist die Rega-Systemvariable `svEnergyCounter...`. Systemvariablen werden von hm-rega nicht ereignisbasiert gemeldet, sondern zyklisch abgefragt.

Einstellung (hm-rega)	Empfehlung
Polling-Intervall	≤ 30 s – deutlich kürzer als das Sendeintervall des ESI

Einstellung (hm-rega)	Empfehlung
Polling-Trigger (optional)	virtuelle Taste, z. B. BidCoS-RF.50.PRESS_SHORT (ID aus dem Objektbaum kopieren)

Mit Polling-Trigger holt hm-rega die Variablen sofort ab, sobald die virtuelle Taste gedrückt wird. Dazu in der CCU ein Programm anlegen:

- **Bedingung:** Gerät HmIP-ESI, Kanal 2 (Zählerstand) *bei Aktualisierung*
- **Aktivität:** Zentrale, virtuelle Taste 50 → *Tastendruck kurz*

Die Taste darf von keinem anderen Programm verwendet werden. Bei 30 s Polling ist der Trigger für die Statistik nicht nötig; er verbessert nur die zeitliche Genauigkeit des Durchflusses.

Kürzere Intervalle kosten Batterielaufzeit (Herstellerangabe > 7 Jahre gilt für die Standardwerte) und erzeugen mehr Quittierungen durch die Zentrale.

10. Kosten und Tarif

Alle Preise werden **netto** hinterlegt, die Mehrwertsteuer weist die Abrechnung getrennt aus. Der Rechenweg entspricht der Systematik einer Gasrechnung:

Energiemenge	= Verbrauch m ³ × Zustandszahl × Brennwert	-> kWh
Arbeitspreis	= kWh × Arbeitspreis ct/kWh ÷ 100	-> € netto
Grundpreis	= Grundpreis €/Monat × (abgerechnete Tage ÷ Tage des Monats)	
Summe netto	= Arbeitspreis + Grundpreis	
Mehrwertsteuer	= Summe netto × MwSt-Satz	
Gesamt brutto	= Summe netto + Mehrwertsteuer	
Saldo	= Abschlag – Gesamt brutto	(positiv = Guthaben)

Jede dieser Zeilen steht mit ihrer konkreten Rechnung in Abrechnung, E-Mail und PDF – also zum Beispiel „422,81 kWh × 8,90 ct/kWh“ statt nur „37,63 €“. Ergänzend werden ausgewiesen: Durchschnittspreis in ct/kWh brutto, Kosten je m³ und Kosten je Tag.

Brennwert und Zustandszahl stehen auf der Jahresrechnung des Lieferanten. Sie schwanken leicht über das Jahr; für eine Kontrollrechnung sind die Vertragswerte ausreichend genau.

Der Grundpreis wird tagesanteilig berechnet. Ein voller Monat ergibt 100 %, ein angefangener Monat entsprechend weniger.

Abschlag: `ABSCHLAG_EUR` ist der monatlich gezahlte Betrag brutto. Die Abrechnung stellt ihn den tatsächlichen Kosten gegenüber und weist Guthaben oder Nachzahlung aus. `Kosten.SaldoJahr` summiert das über das laufende Jahr und wird aus dem Monatsarchiv gebildet – ein erneut gesendeter Bericht verfälscht den Saldo daher nicht.

11. Berichte

Tagesübersicht (Telegram)

Läuft um 00:01 Uhr, direkt nach dem Tagesabschluss, und betrifft den soeben abgeschlossenen Vortag. Inhalt: Verbrauch in m³ und kWh, Kosten, Vergleich zum Vortag mit Trendpfeil, Zählerstand sowie der laufende Monat mit Durchschnitt und Hochrechnung.

Monatsabrechnung (Telegram, E-Mail, PDF)

Läuft am 1. des Monats um 00:10 Uhr für den abgeschlossenen Vormonat. Grundlage sind die zum Monatsanfang gesicherten Zählerstände (`Bericht.Monatsstaende`); fehlen sie, wird auf die Summe der Tageswerte zurückgegriffen. Welche Quelle verwendet wurde, steht in der Abrechnung.

Die Monatsabrechnung enthält:

- Zählerstände zu Beginn und Ende des Zeitraums, Verbrauch und Korrekturfaktor
- Umrechnung in kWh mit Zustandszahl und Brennwert, als nachvollziehbare Formel
- Kostenaufstellung mit Arbeitspreis, Grundpreis, Netto, MwSt und Bruttosumme
- Durchschnittspreis, Kosten je m³ und je Tag, Vergleich zum Vormonat
- Abschlag mit Guthaben oder Nachzahlung
- Tagesverbrauch des Monats als Balkenübersicht

Einrichtung

Adapter	Einstellung im Skript	Hinweis
<code>telegram</code>	<code>TELEGRAM_INSTANZ</code> , <code>TELEGRAM_USER</code>	Nachrichten werden mit <code>parse_mode: HTML</code> gesendet
<code>email</code>	<code>EMAIL_INSTANZ</code> , <code>EMAIL_VON</code> , <code>EMAIL_AN</code>	HTML-Mail, PDF als Anhang
-	<code>PDF_AKTIV</code> , <code>PDF_PFAD</code>	Ordner wird beim ersten Lauf angelegt

Für das PDF muss **pdfkit** in der javascript-Instanz unter *Zusätzliche NPM-Module* eingetragen sein. Nach einem Adapter-Update kann der Eintrag verloren gehen – dann steht im Log ein entsprechender Hinweis und der Versand läuft ohne Anhang weiter.

Die Schriftdateien `PDF_SCHRIFT` zeigen standardmäßig auf DejaVu Sans, das auf den meisten Debian-Systemen vorhanden ist. Ohne diese Dateien nutzt das PDF die eingebaute Helvetica; je nach PDF-Betrachter fehlt dann das hochgestellte ³ in „m³“.

Manuell auslösen und testen

State	Wert	Wirkung
<code>Bericht.TagesberichtSenden</code>	<code>true</code>	Tagesübersicht des zuletzt abgeschlossenen Tages, sofort
<code>Bericht.MonatFuerBericht</code>	<code>2026-08</code>	legt fest, welcher Monat abgerechnet wird; leer = Vormonat

State	Wert	Wirkung
Bericht.MonatsberichtSenden	true	Monatsabrechnung sofort, für den oben gewählten Monat

Wichtig: beim Setzen im Objektbaum den Haken „**bestätigt**“ (**ack**) **nicht setzen** – das Skript reagiert nur auf unbestätigte Werte.

Beides lässt sich beliebig wiederholen; das Monatsarchiv wird je Monat überschrieben statt ergänzt. Wird zum Testen der **laufende** Monat gewählt, landet dessen Zwischenstand im Archiv und damit im Jahressaldo – beim regulären Monatsabschluss wird er durch den endgültigen Wert ersetzt.

Der Tagesbericht setzt mindestens einen abgeschlossenen Tag in der Historie voraus, ist nach dem ersten Tageswechsel also verfügbar. Fehlt für den gewählten Monat jede Datengrundlage, bricht der Monatsbericht mit einem Hinweis im Log ab.

12. Bekannte Grenzen

- **Tagesgrenze.** Die Basis wird mit dem zuletzt gemeldeten Zählerstand gesetzt. Meldet der ESI zuletzt um 23:40 Uhr, zählt der Verbrauch von 23:40 bis 00:00 Uhr zum Folgetag. Bei den üblichen Melderaten liegt der Fehler im Bereich weniger Hundertstel m³ und gleicht sich über die Tage aus.
- **Rollierende Fenster.** Letzte 7/30 Tage summieren die letzten 7 bzw. 30 abgeschlossenen Tage. Die CCU rechnet gleichartig, kann bei einer Neuanlage aber eine längere Vorgeschichte haben – kleine Abweichungen zur WebUI sind in den ersten Wochen normal.
- **Sommer-/Winterzeit.** Alle Datums Grenzen laufen über lokale Zeit; der 25-Stunden- und der 23-Stunden-Tag werden korrekt als ein Tag gewertet.
- **Keine Rekonstruktion ohne History.** Ohne History-Adapter beginnt die Statistik bei null; die CCU-Werte lassen sich nicht rückwirkend importieren.

13. Fehlersuche

Symptom	Ursache / Abhilfe
Log: Quell-State "..." existiert nicht	SRC falsch. Objekt-ID im Objektbaum kopieren
Alle Werte bleiben 0	Quell-State liefert keinen Zahlenwert – Typ prüfen (number , nicht string)
Log: History lieferte keine verwertbaren Daten	Quell-State ist im History-Adapter nicht zur Aufzeichnung aktiviert, oder HISTORY_INSTANCE stimmt nicht
Gestern bleibt 0	Noch kein Tagesabschluss gelaufen – erst am Folgetag gefüllt (oder Vorbefüllung nutzen)
Werte weichen von der CCU ab	INKL_HEUTE prüfen; die CCU zählt den laufenden Tag in den 7-/30-Tage-Werten nicht mit

Symptom	Ursache / Abhilfe
Log: Zählerstand ist gesunken	Zählerwechsel oder Reset der Rega-Variablen – einmalig normal, wiederholt ein Hinweis auf eine instabile Quelle
Historie soll neu aufgebaut werden	<code>Backfill</code> auf <code>true</code> setzen, oder <code>Historie</code> auf <code>[]</code> setzen und Skript neu starten
Durchfluss springt bei Kleinlast zwischen 0 und Wert	<code>NULL_NACH_MIN</code> und <code>LUECKE_MIN</code> zu klein für den Impulsabstand – erhöhen
Durchfluss reagiert träge	<code>FENSTER_MIN</code> verkleinern; CCU-Einstellung A = 0 prüfen (Abschnitt 9)
Durchfluss nach Brennerstart zu niedrig	<code>ANLAUF_MIN</code> an das tatsächliche Sendeintervall anpassen
Log: Ablesung ... ignoriert	Ablesung kleiner als die vorige oder Abweichung über <code>ABLESUNG_MAX_SPRUNG</code> – Wert prüfen, notfalls Grenze anheben
Log: Berechneter Faktor ... unplausibel	Ablesung oder Zeitpunkt passt nicht; alter Faktor bleibt aktiv
Korrekturfaktor bleibt 1	erst ab der zweiten Ablesung und <code>FAKTOR_MIN_VERBRAUCH</code> m ³ Abstand wird er gebildet
Abweichung wächst trotz Korrektur weiter	Faktor über eine längere Periode neu bilden lassen; bei stark schwankendem Verlust Sensorsitz und Zählertyp prüfen
Keine Telegram-Nachricht	<code>TELEGRAM_INSTANZ</code> prüfen; bei gesetztem <code>TELEGRAM_USER</code> muss der Name exakt dem im Adapter registrierten entsprechen
Log: PDF übersprungen – npm-Modul "pdfkit" fehlt	pdfkit in der javascript-Instanz unter <i>Zusätzliche NPM-Module</i> eintragen und Instanz neu starten
Log: PDF-Ordner ... nicht anlegbar	Schreibrechte für den ioBroker-Benutzer auf <code>PDF_PFAD</code> prüfen
Monatsabrechnung zeigt „Summe der Tageswerte“	Für den Monat fehlt ein gesicherter Anfangsstand – ab dem nächsten Monatswechsel ist er vorhanden
Abrechnung erscheint doppelt im Saldo	Kommt nicht vor: das Monatsarchiv wird je Monat überschrieben
Log: Durchfluss ... unplausibel	Sprung im Zählerstand (z. B. Offset nachgetragen) – einmalig normal

Für detaillierte Ausgaben `DEBUG = true` setzen; das Skript protokolliert dann bei jeder Berechnung Zählerstand und alle abgeleiteten Werte.

14. Verwendung in VIS-2

Die States sind mit Rolle und Einheit versehen und lassen sich direkt binden:

```
{0_userdata.0.Gas.Heute}      m³
{0_userdata.0.Gas.Gestern}    m³
{0_userdata.0.Gas.Letzte7Tage} m³
{0_userdata.0.Gas.Letzte30Tage} m³
{0_userdata.0.Gas.Durchfluss} m³/h
{0_userdata.0.Gas.VerbrauchAktiv} true/false – z. B. Flammensymbol ein/aus
{0_userdata.0.Gas.Ablesung}   Eingabefeld für die Ablesung
{0_userdata.0.Gas.AbweichungProzent} %
{0_userdata.0.Gas.Kosten.KostenMonat} €
{0_userdata.0.Gas.Kosten.PrognoseMonat} €
```

Für ein Balkendiagramm der letzten Tage eignet sich der State `Historie` (`role: json`) als Datenquelle – er enthält Datum und Verbrauch je Tag bereits in der Struktur, die die meisten Chart-Widgets erwarten.

15. Quellen

- HmIP-ESI Gas – nicht alle Werte im ioBroker: <https://forum.iobroker.net/topic/82859/hmip-esi-gas-nicht-alle-werte-im-iobroker>
- HmIP-ESI (ioBroker-Forum): <https://forum.iobroker.net/topic/73449/hmip-esi>
- HowTo – erweiterte CCU3/RM Verbrauchszähler in eigene SysVars schreiben: <https://homematic-forum.de/forum/viewtopic.php?t=82352>
- HmIP-ESI Datenpunkte für historische Daten: <https://homematic-forum.de/forum/viewtopic.php?t=81510>
- Metadaten per Script abfragen: <https://homematic-forum.de/forum/viewtopic.php?t=82029>
- CCU: Zyklische Statusmeldungen von Geräten erklärt: <https://technikram.net/blog/2020/04/24/ccu-zyklische-statusmeldungen-von-geraeten-erklaert/>
- HmIP-ESI-GAS Installations- und Bedienungsanleitung (eQ-3): <https://homematic-ip.com/sites/default/files/downloads/hmip-esi-gas-um-web.pdf>
- HmIP-ESI IEC: Intervalle von 6 auf 3 Min. reduzieren: <https://homematic-forum.de/forum/viewtopic.php?t=85378>
- ioBroker.telegram – Dateien und Dokumente senden: <https://github.com/iobroker-community-adapters/ioBroker.telegram>
- ioBroker.email – HTML und Anhänge: <https://github.com/iobroker-community-adapters/ioBroker.email>
- ioBroker.javascript – zusätzliche NPM-Module: <https://github.com/iobroker/ioBroker.javascript/blob/master/docs/de/usage.md>